

Ejercicio 5: Antipatrones de diseño.

Para comenzar con el desarrollo de la actividad primero hay que entender que es un antipatrón de diseño, este es una solución de uso común para un problema habitual que, aunque en una primera instancia parece funcionar o resolver el problema, a largo plazo termina generando más consecuencias negativas como bugs difíciles de rastrear, código difícil de mantener antes que traer beneficios.

Básicamente, son "malas prácticas" disfrazadas de buenas ideas o soluciones que se aplican en contextos equivocados. Conocerlos es tan importante como conocer los patrones reales, porque te enseña qué errores evitar en la arquitectura de software.

Un ejemplo concreto es el "Objeto Dios", que ocurre cuando una sola clase, módulo o componente concentra demasiada lógica, controla demasiada información y hace de todo, violando el principio de Responsabilidad Única, por ejemplo al tener un componente gigante en React que no solo se encarga de renderizar la interfaz de usuario, sino que al mismo tiempo hace los fetches a la base de datos, maneja el estado global de la sesión, formatea las fechas y procesa la lógica de negocio. Si ese componente falla, se cae toda la aplicación.

Otro ejemplo común sería un código espagueti, donde sistemas con un flujo de control tan enredado, carente de estructura y acoplado que es casi imposible seguir la ruta de ejecución sin perderse, por ejemplo archivos de Node.js donde se mezclan llamadas a la base de datos, respuestas HTTP y lógica de negocio en una cadena infinita de funciones anidadas (el famoso *Callback Hell*), sin ningún tipo de abstracción basada en componentes o controladores separados.